

Assignment 1

1. **PUT** is generally used to replace or update an entire resource. The request represents the entire new state of the resource.
PATCH is used for a partial update, where only the fields that need to change are included.

For this scenario, **PATCH** would be more appropriate because the user may only want to change one or two fields, such as the name or password, without replacing the entire data source.

However, if the API specification specifically requires **PUT**, we can still implement **PUT** to support partial updates.

2. "do not change the password" and "change the password to an empty string" are differentiated by whether or not the password field is present in the request. If it is, the password is changed to empty string, while if it is omitted, it doesn't change

Assignment 2

1. The DB has to skip all previous rows to get the requested ones. With large databases, this can slow performance due to the number of rows needed to be skipped.
2. One limitation is that a value like **id** or **created_at** needs to be used to determine where the next page starts. This means we can't jump around arbitrarily.

Assignment 3

1. The 2 requests can make the check at the same time and find the account doesn't exist so a duplicate account might be created.
2. Solution 1: Make a constraint in the DB such that every email has to be unique.
Solution 2: Before checking and inserting, we can make a lock for that email so that any other requests for that email have to wait until the first request finishes.
3. The **UNIQUE** constraint guarantees on the DB level that no two users can have the same email. This doesn't replace the code level check completely as the user is not informed about the email being in use so it would still be useful to have code to inform users of the fact.

Assignment 4

1. Generate new key. keep the old key temporarily, decrypt passwords with old key, re-encrypt them with the new key, update DB with new encrypted vals, test that everything works and get rid of old key.
2. Because Fernet encryption requires the same key that was used for encryption to decrypt the data, so we would still need to decrypt everything with the old key.
3. They should be stored in a secure secrets/key management service like AWS secrets manager.

Assignment 5

1. A conversation may not need a data source if the user is asking general questions or using an AI feature that does not require a specific database.
2. With an `Enum`, adding a new role would require updating the Enum and running a database migration. With a string, we can add new roles without changing the database schema. Strings provide more flexibility for rapid iteration.
3. The complete result should be stored in a database or external storage. Pagination can be used to display a few rows at a time to the frontend.
4. Yes, the JWTs will remain valid until expired. After a password change, we can store the token version or a timestamp of when the password was changed to check for it when validating JWT
5. Move schema introspection to an asynchronous task in the background. The API can give a job ID for the task and the frontend can check periodically for if it is finished.